NEW FACTUS

Manual Técnico

Sistema de punto de venta FACTUS — Esperanza Veliz

Versión software	Laravel 12 · PHP 8.2+
Base de datos	PostgreSQL 14+ (desarrollo) · SQLite (tests)
Audiencia	Desarrolladores y personal de soporte
Idioma de la interfaz	Español (Venezuela)

1. Introducción

NEW FACTUS es una aplicación web de punto de venta (POS) para operar un comercio local. Incluye gestión de productos con presentaciones (Unidad, Mayor, etc.), precios calculados automáticamente desde el costo y el margen, tasas de cambio históricas (BCV/USDT/Promedio), facturación al contado y a crédito (cobrable 100 % en USD), reportes, exportación/importación de datos e impresión térmica. Interactúa además con un *lanzador de escritorio* PowerShell que levanta el servidor local y vigila el proceso ante caídas.

Este manual describe la arquitectura, el modelo de datos, las rutas y flujos de negocio, la instalación/despliegue en equipos cliente y las guías de mantenimiento y resolución de problemas. Para la operación diaria consulte el **Manual de Usuario**.

> **Convención de proyecto:** las cadenas de interfaz, las vistas, los mensajes y los nombres de métodos de prueba están en **español**. Mantenga esta convención en código nuevo.

2. Arquitectura

2.1 Stack de tecnologías

Capa	Tecnología	Observaciones
Backend	Laravel 12 · PHP 8.2+	MVC clásico de Laravel, sin API JSON ni painel de admin genérico; Blade para las vistas.
Base de datos	PostgreSQL (desarrollo) / SQLite en memoria (tests)	.env de desarrollo apunta a `factus_esperanza_veliz`; `phpunit.xml` fuerza SQLite `:memory:`.
Frontend	Bootstrap 5 · jQuery · DataTables 2.x · Alpine.js 3 · Chart.js	Compilado con Vite; Bootstrap para el sistema, Tailwind solo en utilidades del login. DataTables con objetos traducción española inline (ver `layouts/app.blade.php`).
Impresión térmica	mike42/escpos-php	Servicio central `PrinterService`; ver `docs/IMPRESORA.md`.
PDF	barryvdh/laravel-dompdf	Lista de precios y exportación de reportes.
Lanzador de escritorio	PowerShell 5.1 + VBScript	`launcher/FACTUS.vbs` + `factus-launcher.ps1`; watchdog y apagado coordinado.

2.2 Patrones de diseño

- **Service Layer:** la lógica de negocio compleja vive en `app/Services/` (`CatalogoService`, `StockService`, `PrecioService`, `ImpuestoService`, `TasaCambioService`, `PrinterService`, `RegistroNotificaciones`). Los controladores orquestan y persisten.
- **Historial de solo agregar (append-only):** `tasa_cambios` nunca se actualiza; cada cambio inserta una fila nueva. El "valor actual" de un tipo es la última fila por `id`.
- **Snapshot de precios:** al facturar se guarda `tasa_cambio` (decimal) y el detalle de productos en `facturas.productos` (JSON). Cambios posteriores de tasas o precios no alteran facturas existentes; `items_factura` materializa cantidades y precios.
- **RBAC dinámico:** los permisos se asignan a **roles** (tabla `roles`), no a usuarios. El rol `admin` es protegido y siempre tiene todos los permisos.
- **Middleware de autorización:** alias `rol` (CheckRole) y `permiso` (CheckPermiso) registrados en `bootstrap/app.php`. `CheckPermiso` llama `User::hasPermiso($slug)` y responde 403.
- **DataTables server-side:** listados pesados (`/pos/productos`, `/productos/data`, `/productos/ajustar-precios/data`) devuelven `{draw, recordsTotal, recordsFiltered, data}`; búsqueda con `LOWER(columna) LIKE ?` (compatible SQLite + pgsql, nunca `ilike`).
- **Cache de configuración:** `Configuracion::obtener()` lee de cache `file` con clave `configuracion:{clave}`. Todo guardado debe llamar `Configuracion::olvidar()` / `olvidarTodas()`.
- **Notificaciones por contrato:** `app/Services/Notificaciones/` implementa una interfaz `Contracts/Notificacion.php` registrada en `config/notificaciones.php`; evaluación perezosa por petición (no hay cron en la PC del cliente).

2.3 Mapa de carpetas

```
app/
├─ Http/
│  ├─ Controllers/          AjusteController, CategoriaController, ClienteController,
│  │                        DashboardController, FacturaController, HerramientasController,
│  │                        ImpuestoController, LanzadorController, NotificacionController,
│  │                        ProductoController, ProfileController, ReporteController,
│  │                        RolController, TasaCambioController, UserController
│  ├─ Middleware/           CheckRole, CheckPermiso, RegistrarLanzador (alias rol/permiso)
│  └─ Requests/             LoginRequest (login por 'usuario'), request de formularios
├─ Models/                  User, Rol, Permiso, Producto, PresentacionProducto,
│                           Categoria, Cliente, Impuesto, TasaCambio, Factura, ItemFactura,
│                           Configuracion
├─ Services/
│  ├─ Notificaciones/       Contracts/Notificacion.php + Tipos/RecordatorioTasa.php
│  ├─ CatalogoService.php   UMBRAL_STOCK_BAJO=10, METODOS_PAGO
│  ├─ ImpuestoService.php   cálculo de IVA
│  ├─ PrecioService.php     precio original/venta por presentación
│  ├─ PrinterService.php    impresión térmica (ANCHO=48 celdas)
│  ├─ StockService.php      control de inventario
│  └─ TasaCambioService.php helpers de tasas
└─ Support/Moneda.php       formato numérico español n/bs/usd
config/notificaciones.php   registro de tipos de notificación
```

```
routes/web.php + auth.php   definición de rutas
database/migrations/        una migración por tabla (prefijo manual 2026_08_13_*)
database/seeders/           ConfiguracionSeeder, PermisoSeeder, TasaReferenciaSeeder, UserSeeder
launcher/                   Lanzador PowerShell + config.json
resources/views/            vistas Blade en español (layouts/app.blade.php)
```

3. Modelo de datos

3.1 Resumen de tablas y relaciones

- `users` 1—n `facturas` (vendedor); `users.rol` → `roles.slug` (convención + validación `exists:roles,slug`, sin FK).
- `roles` n—m `permisos` vía pivote `permiso_rol` (llave compuesta `rol(slug)+permiso_id`).
- `categorias` 1—n `productos`; `impuestos` 1—n `productos` (nullable).
- `productos` 1—n `producto_presentaciones` (cascade) y 1—n `items_factura`.
- `facturas` 1—n `items_factura` (cascade); `cliente_id`/`user_id` nullable con `nullOnDelete`.
- `facturas.productos` JSON: arreglo de objetos con campos desnormalizados del detalle de la venta (presentación, cantidad, precio, etc.).
- `tasa_cambios`: histórico independiente por `tipo` (sin restricción de unicidad de `tipo`).
- `configuraciones`: par clave→valor (claves `tasa_referencia`, `nombre_negocio`, `moneda`, `imprimir_al_facturar`, `recordatorio_tasa_*`, ...).
- `lanzador_sesiones`: token del lanzador → `session_id` para cierre de sesión controlado.
- Tablas de framework: `sessions`, `cache`, `cache_locks`, `jobs`, `password_reset_tokens`.

3.2 Esquema de tablas de negocio

users

Columna	Tipo	Restricciones	Descripción
id	bigint	PK	—
name	string	NOT NULL	Nombre mostrado
usuario	string	UNIQUE	**Credencial de login** (no email)
email	string	UNIQUE, nullable	Opcional
password	string	NOT NULL	Hash (bcrypt)
rol	string	default 'cajero'	Es el **slug** del rol (ex `roles.slug`, sin FK)
remember_token	string	nullable	Laravel
timestamps	—	—	created_at/updated_at

roles / permisos / permiso_rol

Tabla	Columnas	Notas
roles	id, nombre, **slug(unique)**, descripcion, protegido(bool), timestamps	El slug se autogenera desde `nombre` vía `Str::slug` (+ sufijo `_2` en colisión); inmutable. `protegido=1` (rol admin) no se edita ni elimina.
permisos	id, nombre, **slug(unique)**, descripcion, timestamps	22 permisos precargados (ver 4.2).
permiso_rol	**rol(string)**, permiso_id, timestamps	PK compuesta (rol, permiso_id). `rol` guarda el slug.

categorias / impuestos / clientes / configuraciones

Tabla	Columnas	Notas
categorias	id, nombre(unique 100), timestamps	—
impuestos	id, nombre, porcentaje(decimal 5,2), timestamps	Ej. IVA 16.
clientes	id, **ci(unique)**, nombre, telefono(null), timestamps	Referenciado por facturas (nullOnDelete). Se puede crear "rápido" desde el POS.
configuraciones	id, **clave(unique 50)**, valor(string 255), timestamps	Claves: `tasa_referencia`, `nombre_negocio`, `moneda`, `imprimir_al_facturar`, `recordatorio_tasa_activo`, `recordatorio_tasa_hora1/2`, ...

productos

Columna	Tipo	Descripción
id	bigint PK	—
categoria_id	FK null	→ categorias, nullOnDelete
nombre	string	Nombre único de negocio
descripcion / imagen	text / string null	Opcionales
stock_actual	decimal(14,4)	Conteo manual o vía ajustes
unidad_medida	string default 'unidad'	—
impuesto_id	FK null	→ impuestos, nullOnDelete
costo_usd	decimal(12,2)	—
estado	string default 'disponible'	'disponible'/'desactivado'
deleted_at	timestamp null	Soft deletes (ex-producto desactivado)

> La columna original `fuente_tasa` de productos se movió a presentaciones en la migración `2026_08_14_000001`. No reintroducirla en el modelo.

producto_presentaciones

Columna	Tipo	Descripción
id	bigint PK	—
producto_id	FK	→ productos, cascadeOnDelete
nombre	string	Ej. "Unidad", "Mayor"
factor_conversion	decimal(14,4)	Unidad=1
margen	decimal(5,2)	Margen % de la presentación
precio_usd	decimal(12,2)	Precio calculado
activa	boolean	Presentación visible en POS
fuente_tasa	string default 'promedio'	bcv/usdt/promedio por presentación (desde 2026_08_14)

tasa_cambios

Columna	Tipo	Descripción
id	bigint PK	Orden cronológico real (tie-break de historial)
user_id	FK null	→ users, nullOnDelete
tipo	string	código interno (bcv/usdt/promedio u otros), autogenerado del nombre vía Str::slug
nombre	string null	Nombre visible; fallback `ucfirst($tipo)`
monto	decimal(12,2)	—
fecha	date	Índice (tipo,fecha)
activo	boolean	Toggle por toda la serie del tipo
origen	string default 'manual'	'manual'|'importado'
timestamps	—	—

Importante: NO hay unicidad sobre `tipo`. La tasa vigente para un tipo es `TasaCambio::ultimaDe()` (mayor `id`), jamás `latest('created_at')`.

facturas

Columna	Tipo	Descripción
id	bigint PK	—
correlativo	string UNIQUE	Número de control del ticket

cliente_id / user_id	FK null	nullOnDelete
productos	json	Snapshot del detalle de la venta
tasa_cambio	decimal(12,2)	Snapshot de la tasa al facturar
metodo_pago	string	efectivo|punto|biopago|divisas|pago_movil|transferencia|mixto|**credito** (centinela)
detalle_pago	json null	Desglose de pagos mixtos
subtotal_bs / iva_bs / total_bs	decimal(14,2)	Totales en Bs
total_usd	decimal(12,2)	Equivalente USD (deuda fija en créditos)
pago_bs / fecha_pago	decimal / timestamp null	Únicas columnas de crédito; se llenan al cobrar
estado	string default 'contado'	contado|credito|anulado
estado_credito	string null	'pendiente'|'cancelado'
fecha_venta	dateTime	Índice; usada por reportes de filas

items_factura

Columna	Descripción
factura_id (FK)	→ facturas, **cascadeOnDelete**
producto_id (FK)	→ productos, cascadeOnDelete
presentacion_id / presentacion_nombre	Referencia de presentación (null permitido)
factor_conversion	decimal(14,4) default 1
cantidad	decimal(14,4)
precio_unitario_usd / precio_unitario_bs	Precios aplicados
subtotal / unidad_medida	—

lanzador_sesiones

Columna	Descripción
token (PK)	Token del lanzador (archivo en %LOCALAPPDATA%\FACTUS\token.txt)
session_id	ID de la sesión web a destruir
user_id	→ users (índice)

4. Rutas, permisos y lógica de negocio

4.1 Mapa de rutas (routes/web.php)

Método	URI	Controlador@método	Permiso requerido
GET	/	redirect → login	—
GET	/dashboard	DashboardController@index	ver-dashboard
POST	/lanzador/cerrar-sesion	LanzadorController@cerrarSesion	— (exento CSRF, autenticación implícita por sesión vinculada)
Grupo auth			
GET/PATCH/DELETE	/profile	ProfileController	auth
GET	/notificaciones/pendientes	NotificacionController@pendientes	auth
POST	/notificaciones/{tipo}/posponer	NotificacionController@posponer	auth
GET	/productos/data	ProductoController@data (DataTables server-side)	gestionar-productos
GET/POST/...	/productos (resource, sin show)	ProductoController	gestionar-productos
POST	/productos/{id}/restore	ProductoController@restore	gestionar-productos
GET	/productos/ajustar-precios (+ /data)	AjusteController@editarPrecios/@preciosData	actualizar-precios
POST	/productos/{producto}/ajustar-precio	AjusteController@guardarPrecio	actualizar-precios
GET	/productos/ajustar-inventario	AjusteController@editarInventario	actualizar-inventarios
POST	/productos/{producto}/ajustar-inventario	AjusteController@ajustarInventario	actualizar-inventarios
GET/POST/...	/clientes (resource, sin show)	ClienteController (+ storeRapido en POST /clientes/rapido)	gestionar-clientes
GET/POST/...	/impuestos (resource, sin show)	ImpuestoController	gestionar-impuestos
GET	/tasas-cambio	TasaCambioController@index	gestionar-tasas
GET	/tasas-cambio/historial	TasaCambioController@historial (paginado 20)	gestionar-tasas
POST	/tasas-cambio	TasaCambioController@store (nuevo tipo)	gestionar-tasas
POST	/tasas-cambio/actualizar	TasaCambioController@actualizar (inserta fila)	gestionar-tasas
POST	/tasas-cambio/referencia	TasaCambioController@fijarReferencia	gestionar-tasas
POST	/tasas-cambio/{tasa}/toggle	TasaCambioController@toggleEstado	gestionar-tasas
GET/POST/...	/categorias (resource, sin show)	CategoriaController (+ asignar-productos)	gestionar-categorias
GET	/pos	FacturaController@pos	usar-pos
GET	/pos/productos	FacturaController@posProductos (DataTables server-side)	usar-pos
POST	/facturas	FacturaController@store (crear factura)	crear-facturas
GET	/facturas	FacturaController@index	ver-facturas
GET	/creditos	FacturaController@creditos	gestionar-creditos
GET	/facturas/{factura}	FacturaController@show (recibo + reimpresión)	ver-facturas
POST	/facturas/{factura}/pagar-credito	FacturaController@pagarCredito	gestionar-creditos
POST	/facturas/{factura}/anular	FacturaController@anular	anular-facturas
GET	/reportes/facturas	ReporteController@facturas (+ queryFacturas)	ver-reporte-facturas
GET	/reportes/estadisticas	ReporteController@estadisticas	ver-reporte-facturas
GET	/reportes/balance	ReporteController@balance (?anio=...)	ver-balance

GET	/reportes/stock	ReporteController@stock	ver-stock-bajo
GET	/herramientas/datos + /exportar	HerramientasController@datos/@exportar	exportar-datos
POST	/herramientas/importar	HerramientasController@importar	importar-datos
GET/POST	/herramientas/impresora (+ /test)	HerramientasController@imprimirConfig/@imprimirGuardar/@imprimirTest	configurar-impresora
GET	/herramientas/imprimir-factura/{factura}	HerramientasController@imprimirFactura	configurar-impresora
GET	/herramientas/precios (+ /pdf, /imprimir)	HerramientasController@precios/@preciosPdf/@imprimirPrecioProducto	ver-lista-precios
GET/POST	/herramientas/configuracion	HerramientasController@configuracion/@configuracionGuardar	configuracion
POST	/herramientas/recordatorio	HerramientasController@recordatorioGuardar	configuracion
GET/POST/...	/usuarios (resource, sin show)	UserController	gestionar-usuarios
GET/POST/...	/roles (resource, sin show)	RolController	gestionar-usuarios

4.2 Autenticación y RBAC

- **Login por usuario:** `LoginRequest` valida `usuario`/`password` (campo `usuario`, no email). El seeder inicial crea `admin`/`admin123`.
- **Middlewares:** `auth` + `permiso:{slug}`. `CheckPermiso` consulta `User::hasPermiso($slug)`; 403 si no aplica. `RegistrarLanzador` corre antes que `auth` y registra el token del lanzador por petición.
- **Rol admin protegido:** `protegido=true`; siempre sincronizado con todos los permisos. No se edita ni elimina. Al borrar un rol, se bloquea si hay usuarios referenciándolo.
- **Usuarios:** un usuario NO tiene permisos propios; recibe todo de su rol. El rol se valida como `exists:roles,slug`.
- **Lista de 22 permisos** (PermisoSeeder):

```
ver-dashboard, usar-pos, gestionar-productos, gestionar-clientes, ver-facturas,
crear-facturas, anular-facturas, gestionar-creditos, gestionar-impuestos,
gestionar-tasas, gestionar-categorias, gestionar-usuarios, ver-reporte-facturas,
ver-balance, ver-stock-bajo, exportar-datos, importar-datos, configurar-impresora,
ver-lista-precios, configuracion, actualizar-precios, actualizar-inventarios
```

4.3 Flujo de venta en POS (FacturaController@store)

1. Valida el payload del carrito (productos por id de presentación, cantidades, método de pago).
2. Dentro de una **transacción DB**: recalcula precios desde `PrecioService` (tasa vigente por la `fuente_tasa` de cada presentación), aplica IVA con `ImpuestoService`, calcula total Bs y USD.
3. **Descuenta stock** con `StockService` (`lockForUpdate` cuando aplica para evitar carreras) y guarda factura + `items_factura` + snapshot JSON.
4. **Crédito:** si `metodo_pago=credito` el total es 100 % USD (`total_usd`), `estado=credito`, `estado_credito=pendiente`; se fuerza el valor centinela `'credito'`.
5. **Mixto:** `detalle_pago` guarda el desglose (efectivo + punta/biopago/...).
6. Tras `DB::commit()`, imprime el ticket en `try/catch` si `imprimir_al_facturar='1'`. La venta nunca falla por la impresora; la respuesta JSON del POS incluye `impreso` y `impresion_error`.

4.4 Créditos (facturas a crédito)

- La deuda queda fija en `total_usd` (100 % USD). No se elige método de pago al vender; `store()` fuerza `credito`.
- **Cobro** (`pagarCredito`): actualiza la MISMA fila; requiere método (`in:efectivo,punto,biopago,divisas,pago_movil,transferencia`, mixto rechazado); pone `metodo_pago` real, `estado_credito='cancelado'`, `pago_bs = total_usd × tasa vigente` y `fecha_pago`. No existe `pago_usd`: `total_usd` funciona como monto pagado.
- "A pagar hoy (Bs)" se muestra solo en la pantalla de cobro (`facturas.creditos`) como `total_usd × tasaVigente` (por `ultimaDe()`).
- **Anulación:** bloqueada cuando `estado_credito='cancelado'`. `anular()` reingresa stock vía `StockService` en facturas no anuladas.

> **Nuance de reportes (comportamiento documentado, no un bug):** el ingreso de un crédito cobrado cuenta el día del cobro (`fecha_pago`) en KPIs/desglose/Estadísticas/Balance (vía `Factura::ingresosEn()`), mientras que la lista/PDF/CSV de `reportes/facturas` filtra por `fecha_venta`

> y muestra la fila el día en que se emitió (badge "Crédito", `total_bs`). Si se cambia un día, decidir ambos lados consistentemente y mantener CSV/PDF en sincronía.

4.5 Tasas de cambio

- `actualizar()` **nunca actualiza**: inserta fila nueva por cambio con `user_id` y `origen='manual'`. `store()` crea un tipo nuevo (slug del nombre, con sufijo `_2`… en colisión).
- La "actual" de un tipo = última fila por `id`: `TasaCambio::ultimaDe()`, `activade()`, `ultimasPorTipo()`, `mapaMontos()`.
- La referencia (Configuracion `tasa_referencia`, default `'bcv'`) se fija solo en una tasa activa (`fijarReferencia`); `toggleEstado()` volca `activo` de la serie completa y está bloqueado cuando el tipo es la referencia actual.
- **Historial:** `orderByDesc('created_at')->orderByDesc('id')` (el tie-break por `id` importa, tests insertan en el mismo segundo), paginado 20, `variacion` contra la fila anterior del Mismo tipo.
- Las facturas guardan snapshot de tasa; el POS divide totales USD por la **referencia**. La importación inserta con `origen='importado'`.

4.6 Productos, presentaciones y precios

- Un producto puede tener varias presentaciones (factor de conversión), cada una con su `margen` y `fuente_tasa`.
- `PrecioService::precioPresentacion(costo, margen, factor)` calcula el precio de venta. `stock_actual` es conteo propio; los ajustes (`AjusteController`) y la venta modifican stock.
- Productos desactivados no se editan (`edit/update` cargan `withTrashed()` y redirigen; el botón Editar se oculta en filas trashed); se restauran con `POST /productos/{id}/restore`.

4.7 Reportes

Ruta	Contenido
/reportes/facturas	Tabla server-side del Reporte de Ventas con `queryFacturas()` (ver nuance en 4.4), filtros por rango/fecha, export PDF y CSV.
/reportes/estadisticas	KPIs y gráficas Chart.js (ingresos, productos más vendidos, etc.) usando `Factura::ingresosEn()`.
/reportes/balance	Ingresos por mes (con `?anio=`), también `ingresosEn()`.
/reportes/stock	Lista de productos con stock bajo (umbral `CatalogoService::UMBRAL_STOCK_BAJO` = 10).

4.8 Herramientas

- **Exportar/Importar:** layout JSON. `importar()` valida por producto (nombre) y crea/actualiza; acepta también layouts legacy (fuente_tasa a nivel producto → fallback). Verifica `rol` string bruto en el controlador.
- **Lista de precios** (`precios`/`preciosPdf`): filtros `categoria_id` y `presentacion[]` (validados contra presentaciones activas; inexistentes se ignoran; sin filtro = todas). El `export=json` y el PDF aplican el mismo criterio; el PDF añade sufijo al nombre del archivo.
- **Impresora:** `PrinterService::configuracion()` lee `storage/app/impresora.json`; test de impresión y reimpresión de factura.
- **Configuración:** clave/valor del negocio (nombre, moneda, tasa referencia, recordatorio de tasas, IVA por defecto, etc.).

4.9 Notificaciones programadas

- Contrato: implementar `Contracts/Notificacion.php` (`tipo`, `permisoRequerido`, `debeMostrar(Carbon)`, `titulo`, `mensaje`, `accionUrl`, `textoAccion`, `posponerHasta`) y registrar en `config/notificaciones.php`.
- Widget Alpine (`campanaNotificaciones`) consulta `GET /notificaciones/pendientes` cada 60 s; "Recordar más tarde" usa `POST /notificaciones/{tipo}/posponer`.
- Ejemplo actual: `RecordatorioTasa` avisa a quienes tienen `gestionar-tasas` si la referencia no se actualizó en la ventana (`recordatorio_tasa_activo`, horas `recordatorio_tasa_hora1/2`, default 09:00/14:00).

4.10 Lanzador de escritorio

- `launcher/FACTUS.vbs` → `factus-launcher.ps1` (consola oculta). Levanta `php artisan serve --host=0.0.0.0 --port=8000`, abre el navegador en modo app; al cerrar la ventana: mata navegador, `POST /lanzador/cerrar-sesion` y mata PHP (sin procesos huérfanos).
- Verifica PostgreSQL (puerto 5432, TCP) y ejecuta un **watchdog**: si el puerto del servidor cae, reinicia PHP (hasta 3 intentos) y recarga la página (F5).
- Config: `launcher/config.json` (`port`, `host` — default `0.0.0.0` para acceso LAN por IP, `phpPath`, `appPath`, `browser`, `postgresPort`). Estado en `%LOCALAPPDATA%\FACTUS\`.

- Cierre de sesión controlado vía `lanzador_sesiones` y middleware `RegistrarLanzador` (prepended antes de `AuthenticatesRequests`).

4.11 Migración del sistema viejo

`php artisan migrar:inventario-viejo --archivo="ruta.sql" [--force] [--export-json="ruta.json"]` parsea INSERT de la tabla `inventories` (15 columnas, Latin1/Win-1252 auto-detectado, BOM UTF-8 tolerado). Detalles:

- Nombre de producto = combinación deduplicada de `description` + `name` (tokens repetidos una vez, se quita la palabra "Mayor"). La clave de agrupación usa el nombre normalizado para emparejar normal↔Mayor.
- Costo = inversa `round(precie_usd × (1 − porcentage/100), 2)`; categoría/imagen/impuesto vacíos; `stock_actual=0`.
- Presentaciones: Unidad (factor 1) y/o Mayor (factor = `units_package`); `fuente_tasa` mapea `daily_dollar` (0→bcv, 1→usdt, 2→promedio).
- `--force` trunca producto_presentaciones, productos, categorías e impuestos antes de re-ejecutar; sin `--force` es idempotente (salta productos existentes por nombre).

5. Instalación y despliegue

5.1 Requisitos

Componente	Versión	Notas
PHP	8.2+ (probado con 8.3)	extensiones: pdo_pgsql/pdo_sqlite, gd (cap. vis), zip, xml
Composer	2.x	—
Node + npm	20+ / 10+	para assets Vite; no requerido en PC cliente ya desplegada
PostgreSQL	14+ (dev)	en tests se usa SQLite en memoria
Navegador	Edge/Chrome/Brave	usado por el lanzador (modo app)
Git	opcional	solo desarrollo; despliegues cliente no llevan Git

5.2 Variables de entorno (.env)

Variable	Dev (pgsql)	Tests	Descripción
APP_ENV / APP_DEBUG	production / false	testing / true	El .env dev es intencionalmente "production" (no "arreglarlo")
APP_NAME	NEW FACTUS	—	Marca visual
APP_URL	http://localhost	http://localhost	—
DB_CONNECTION	pgsql	sqlite :memory:	`phpunit.xml` fuerza SQLite
DB_DATABASE	factus_esperanza_veliz	—	Host 127.0.0.1:5432, user postgres
SESSION_DRIVER / CACHE_STORE / QUEUE_CONNECTION	database	database/array	Cache file en configuraciones

5.3 Instalación desde cero (desarrollo)

```
composer install
copy .env.example .env          # ajustar DB_* a PostgreSQL local
php artisan key:generate
php artisan migrate --seed
npm install
npm run build
php artisan serve               # o: composer dev (server + queue + pail + vite)
```

> El seed levanta: admin/admin123 (UserSeeder), permisos + rol admin (PermisoSeeder), configuraciones del negocio (ConfiguracionSeeder) y tasas iniciales bcv/usdt/promedio en 0 con referencia bcv (TasaReferenciaSeeder).

5.4 Despliegue en equipo cliente (Windows, sin Git/Node)

1. Instalar PostgreSQL 14+ y PHP 8.2+ (configuración manual detallada en `docs/INSTALACION.md`).
2. Copiar la carpeta de la app (sin `node_modules` / `.git`) a la PC cliente; **no sobrescribir** `.env` ni `launcher/config.json` en actualizaciones.
3. Crear la base y ejecutar `php artisan migrate --force` (con `--seed` la primera vez).
4. Configurar `launcher/config.json` (`host` vacío/127.0.0.1 para local-dev, o IP del equipo para red).
5. Acceso desde otras PCs: regla entrante en Firewall (`netsh advfirewall firewall add rule name="FACTUS" dir=in action=allow protocol=tcp localport=8000`) y abrir `http://IP-DEL-EQUIPO:8000`.
6. Doble clic en `FACTUS.vbs` para arrancar; al cerrar la ventana se apaga todo.

5.5 Pruebas

```
php artisan test                 # toda la suite (SQLite en memoria)
php artisan test --filter=Nombre  # un test específico
```

- Casi toda ruta está tras el middleware `permiso:{slug}`; los tests Feature siembran `PermisoSeeder` y usan `User::factory()->create(['rol' => 'admin'])` + `actingAs()`.
- En `FacturaControllerTest` el `setUp` desactiva `imprimir_al_facturar` para no depender de la impresora.
- Para tasas "viejas": `$tasa->forceFill(['created_at' => …])->save()` (update de `created_at` se descarta por `$fillable`).
- Antes de publicar cambios: ejecutar `vendor\bin\pint` sobre los archivos tocados.

6. Mantenimiento y módulos

6.1 Agregar un permiso nuevo

1. Añadir la entrada `['nombre' => …, 'slug' => …]` al arreglo de `PermisoSeeder`.
2. Proteger la(s) ruta(s) con `->middleware('permiso:mi-slug')`.
3. Ejecutar el seeder en dev para que el rol admin obtenga el permiso (con `firstOrCreate` + `sync` es seguro repetirlo).

6.2 Agregar un tipo de notificación

1. Crear clase en `app/Services/Notificaciones/Tipos/` que implemente `Contracts/Notificacion`.
2. Registrarla en `config/notificaciones.php`. Nada más: rutas, registro, widget ya lo recogen.
3. Si configura el usuario vía pantalla (interruptor/horas), persistir con `Configuracion` + `olvidar()` la clave.

6.3 Página nueva con DataTables server-side

1. Endpoint que devuelve `{draw, recordsTotal, recordsFiltered, data}`.
2. Búsqueda con `LOWER(columna) LIKE ?` (compatible SQLite + pgsql).
3. Vista con `init DataTables` y `window.DataTableSpanish` (objeto literal, NO `url: '/js/i18n/…'` asíncrono — rompe DataTables 2.3.8 con `order` + `columnDefs` "Ningún dato disponible").
4. No embeker colecciones grandes en la vista.

6.4 Formato numérico

- Backend: `\App\Support\Moneda` (`n($v, $dec=2)`, `bs()`, `usd()` → es-VE). Frontend: `window.fmtMoneda` (Intl es-VE) en `resources/js/app.js`.
- Excepciones deliberadas: inputs `type="number"` (punto decimal), aritmética interna del POS (compara crudo `diferenciaPagos > 0.01`), CSV de Reportes y clave interna de StockService tienen formato propio.

6.5 Migraciones y cache

- Usar prefijo de fecha manual, p.ej. `2026_09_01_000000_nombre.php`. El esquema fue reconstruido (squash) en la tanda `2026_08_13_*`; no crear backfills legacy.
- Toda escritura de Configuración debe llamar `Configuracion::olvidar($clave)` (o `olvidarTodas()`). Para buscar claves en seeders usar `where('clave')->exists()`, nunca `obtener()` (evita leer cache).
- Al desplegar cambios de esquema en cliente: `php artisan migrate --force`.

6.6 Impresión térmica

- Central: `PrinterService` (`ANCHO=48` celdas para 80 mm). `itemsDesdeFactura(Factura)` es el builder compartido (usa `posiciones`/`pagos`).
- Todo texto de usuario se imprime con `Str::ascii()` (1 char = 1 celda). Columnas dinámicas por fila; totales a doble tamaño con fallback `(1,2)`.
- Etiqueta de precio: negocio omitido, producto 2x negrita centrado (24 celdas), precio 2x2 (tamaño 4 si ≤12 chars).
- `imprimir_al_facturar` (default '1') controla la impresión automática post-commit.

6.7 Manejo de errores en la UI

- Flujos que pueden fallar redirigen con `withErrors(['error' => …])` (se muestra como toast en el layout), nunca 404/500 crudos.
- Guards vigentes: producto desactivado no editable; impuesto con productos / cliente con facturas / último admin / auto-eliminación no eliminables; `pagarCredito` sin tasa referencia redirige con aviso (botón Cobrar deshabilitado sin `$tasaVigente`).
- Páginas de error propias (español) en `resources/views/errors/{403,404,500,503}.blade.php`.

7. Resolución de problemas (Troubleshooting)

Síntoma	Causa posible	Solución
403 al entrar a un módulo	El rol del usuario no tiene el permiso; o rol no sincronizado	Revisar rol en `/roles`; re-correr `PermisoSeeder`. El rol admin nunca da 403.
POS "Sin tasa disponible" / botones deshabilitados	No hay tasa activa o sin `tasa_referencia`	Ingresar/activar tasas en `/tasas-cambio`; `fijarReferencia`.
Impresora no imprime	Config en `storage/app/impresora.json` incorrecta, o impresora apagada	Test en `/herramientas/impresora`; revisar `docs/IMPRESORA.md`; la venta no se pierde (impresión es post-commit).
DataTables muestra "Ningún dato disponible" en pantallas con orden	Se cambió `DataTableSpanish` a carga asíncrona i18n	Revertir a objeto literal español (ver 6.3).
Puerto 8000 ocupado	Otra instancia PHP/Laravel en ejecución	Matar proceso; revisar `%LOCALAPPDATA%\FACTUS\php.pid`; el watchdog del lanzador reinicia el servicio.
Otra PC no accede por IP	Firewall sin regla entrante, o host de servidor no es 0.0.0.0	Aplicar regla `netsh advfirewall` (5.4) y verificar `launcher/config.json`.
Crédito cobrado no aparece en Reporte de ese día	Comportamiento documentado: filas por `fecha_venta`, ingreso por `fecha_pago`	Buscar la factura por fecha de emisión; el ingreso aparece en KPIs/estadísticas el día del cobro.
Config no parece aplicar (cambia por un momento)	Cache de configuración sin invalidar	Verificar `olvidar()` tras guardado; borrar `storage/framework/cache`.
Login falla pese a credenciales correctas	Rate limit (5 intentos) o sesión previa	Esperar y reintentar; el admin puede revisar `/usuarios`.
Editar producto desactivado no funciona	Guardián de soft-deletes	Restaurar primero (`POST /productos/{id}/restore`).
Después de actualizar, faltan columnas/tablas	Migraciones no corridas en la PC cliente	`php artisan migrate --force` (instrucciones en `docs/INSTALACION.md` §8).

Errores comunes del importador viejo

- "Archivo no válido": verificar ruta del SQL, encoding (Latin1/Win-1252) y que sea de la tabla `inventories`.
- Productos duplicados al re-importar sin `--force`: es el comportamiento idempotente esperado (omite existentes por nombre).
- Usar `--export-json` para previsualizar/editar el catálogo sin tocar la BD antes de importar por la UI.